

A Bus Driver Scheduling Problem: a new mathematical model and a GRASP approximate solution

Renato De Leone · Paola Festa · Emilia Marchitto

Received: 18 May 2005 / Revised: 29 March 2010 / Accepted: 8 July 2010 /

Published online: 22 July 2010

© Springer Science+Business Media, LLC 2010

Abstract This paper addresses the problem of determining the best scheduling for Bus Drivers, a $\mathcal{NP}$ -hard problem consisting of finding the minimum number of drivers to cover a set of Pieces-Of-Work (POWs) subject to a variety of rules and regulations that must be enforced such as spreadover and working time. This problem is known in literature as Crew Scheduling Problem and, in particular in public transportation, it is designated as Bus Driver Scheduling Problem. We propose a new mathematical formulation of a Bus Driver Scheduling Problem under special constraints imposed by Italian transportation rules. Unfortunately, this model can only be usefully applied to small or medium size problem instances. For large instances, a Greedy Randomized Adaptive Search Procedure (GRASP) is proposed. Results are reported for a set of real-word problems and comparison is made with an exact method. Moreover, we report a comparison of the computational results obtained with our GRASP procedure with the results obtained by Huisman et al. (Transp. Sci. 39(4):491–502, 2005).

Keywords Crew and Bus Driver Scheduling Problem · Transportation · Meta-heuristics · GRASP

R. De Leone · E. Marchitto

School of Science and Technology, University of Camerino, Via Madonna delle Carceri, 9,
62032 Camerino, MC, Italy

R. De Leone

e-mail: renato.deleone@unicam.it

E. Marchitto

e-mail: emilia.marchitto@unicam.it

P. Festa (✉)

Department of Mathematics and Applications, University of Napoli Federico II, Naples, Italy

e-mail: paola.festa@unina.it

Introduction

The Bus Driver Scheduling Problem (BDSP) is an extremely complex part of the Transportation Planning System (e.g., Wren 2004; Desaulniers and Hickman 2003; Mesquita et al. 2009; Portugal et al. 2009; Moz et al. 2009). Its combinatorial nature and the need to solve large size real-world problems has led to the development of several heuristics. Wren and Rousseau (1995) give an outline of the BDSP and propose various approaches for solving it. Many of these techniques have been reported in the proceedings of international conferences on Computer-Aided-Scheduling of Public Transport (e.g., Wren 1981; Rousseau 1985; Daduna and Wren 1988; Desrochers and Rousseau 1992; Daduna et al. 1995; Wilson 1999; Voß and Daduna 2001).

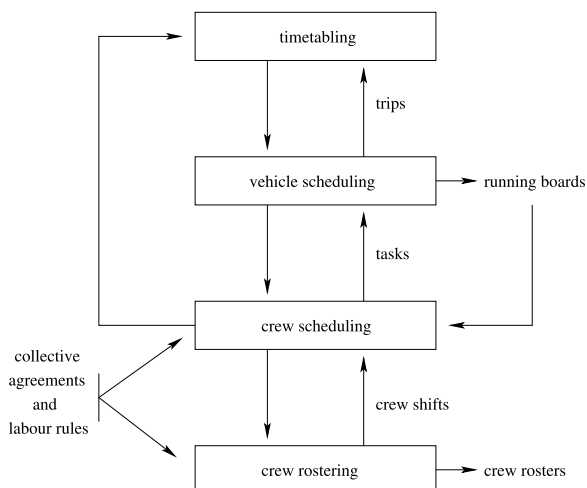
In general, the Transportation Planning System is divided in different subproblems due to its complexity: Timetabling, Vehicle Scheduling, Crew Scheduling (Bus and Driver Scheduling), and Crew Rostering (see Fig. 1 for the relationship between these subproblems).

The transportation service is composed of a set of lines that corresponds to a bus travelling between two locations of the same city or between two cities. For each line, the frequency is determined by the demand. Then, a timetable is constructed, resulting in journeys characterized by a start and end point, and a start and end time. The Vehicle Scheduling Problem consists in finding a schedule for the buses, each schedule being defined as a bus journey starting at the depot and returning to the same depot.

The objective is to minimize the total cost given by the cost of buses used for the service and running costs. Running costs can be minimized avoiding unnecessary *deadheads*, i.e., trips carrying no passengers. The daily schedule of each single bus is known as a *running board* (or vehicle block).

Bus schedules are composed of several running boards and their lengths are determined by the total time the bus is operating away from the depot.

Fig. 1 Transportation Planning System



The Bus Driver Scheduling is the second phase of the general operational planning of a public transportation company. This problem is important from an economic point of view and it is related to the costs of the drivers. The BDSP consists of determining the shifts (i.e., full days of work) for the drivers at a certain depot to cover all the running boards.

Since a driver exchange can occur at various points along a running board, the entire running board is divided in units (*Pieces-Of-Work*) that start and finish at *relief points*, i.e., designed locations and times where and when a change of driver may occur. Different types of shift can be taken into account, which are composed of Pieces-Of-Work whose length is variable and whose beginning occurs at possibly distinct starting times. For example, a shift can contain a single Piece-Of-Work, two Pieces-Of-Work or more Pieces-Of-Work.

A set of Pieces-Of-Work that satisfies all the constraints is a *feasible shift*. A break (a time slot of no work) can also be inserted between two different Pieces-Of-Work or in a single Piece-Of-Work (i.e., when the vehicle is stopped for a period of time at a location where there is not a relief point). There are two different types of break: *rest* and *idle time*. Rest is unpaid, while idle time is paid (in Italy, for example, its rate is 12 percent of ordinary wage).

The feasibility of a shift not only depends on the breaks and the duration of the Pieces-Of-Work, but also on the *total working time* and on the *spreadover*. The total working time is the sum of the duration of the Pieces-Of-Work (excluding idle time) while the spreadover is the total time between the start and the end of the shift.

A feasible solution for the BDSP is a set of feasible shifts for the drivers. A different cost to each shift can be associated. The aim is to minimize not only the total cost but also the total number of shifts.

In Fig. 2 an example of a vehicle schedule is represented. It may typically be depicted by several horizontal lines, each representing a running board. Along these lines the relief points are identified by a time/location pair, at which drivers can be relieved. For example, the pair (A, 08:10) is designed as relief point, where A is the relief location and 08:10 is the relief time. An example of Piece-Of-Work is the interval between the pairs (B, 15:40) and (A, 16:10) that, in general, must be covered

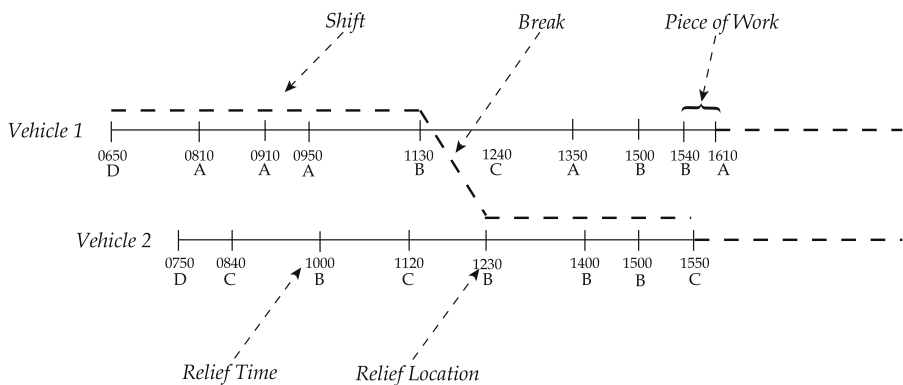


Fig. 2 Representation of vehicle schedule and of a possible shift

by a single driver. Figure 2 shows a part of the schedule of Vehicle 1 that leaves the depot at 06:50 and passes relief point A at various times, relief point B at 11:30, and so on. Moreover, an example of a possible shift is also shown including the first part of Vehicle 1, a break, and the second part of Vehicle 2 (dashed line connecting the timetable line for Vehicle 1 to the timetable line for Vehicle 2).

The BDSP which is a $\mathcal{NP}$ -hard problem even when there are only spreadover and working time constraints (for a proof see Fischetti et al. 1987, 1989), can be formulated as a Set Partitioning Problem or a Set Covering Problem. By solving the Set Covering Problem we often produce a solution that contains very little or no over-covering at all. Moreover, the over-covering can be eliminated *a posteriori* using, for example, a heuristic procedure.

Given the Set Covering Problem, different heuristic approaches such as column generation, Lagrangian and Linear programming relaxation are used to solve the BDSP.

Several heuristic approaches have been proposed in the literature; for a survey see Wren and Rousseau (1995) and Wren (2004); Mesquita et al. (2009); Portugal et al. (2009); Moz et al. (2009).

Curtis et al. (1999) solved the restricted Set Partitioning model by means of a hybrid constraint programming/linear programming heuristic method, where the linear programming solutions are adopted to guide variable and value ordering in the constraint programming algorithm. One interesting aspect of this method is that the constraint programming component is able to deal with nonlinear constraints which can derive from specific work rules.

Recently, Fores et al. (2002) combined a column generation strategy with the set covering model. The approach consists of generating new columns as needed from a subsets of *a priori* enumerated valid shifts. This strategy is applied only to the root node of the Branch & Bound search tree. The solution is significantly improved with a slight increase in the solution time.

Column generation techniques for the BDSP were introduced by Desrochers and Soumis (1989) and they have been successfully tested on real-world problems (e.g., Rousseau and Desrosiers 1993). The problem is decomposed into two subproblems: a Set Covering and a Shortest Path Problem. By solving the Set Covering subproblem, a schedule from already known feasible shifts is determined. Starting from this newly found schedule, a Shortest Path Problem is solved to find a new and better feasible set of shifts.

Carraresi et al. (1993) propose a decomposition approach based on Lagrangian relaxation that can also be applied to the Airline Crew Scheduling Problem.

Dias et al. (2001) proposed a hybrid relaxation of the Set Partitioning model and a Genetic Algorithm; recently, in Dias et al. (2005), they proposed a new multi-objective Genetic Algorithm based on a Pareto approach. Furthermore, they have depicted a set of novel operations: a fitness assignment procedure based on the *dominance sharing* notion, a coding scheme and several new operators. The approach has been tested on a set of real-world problems from three mass transportation companies.

Finally, various papers describe commercial software packages used by public transport companies: the HASTUS system, which has been on the market for more

than 20 years (for more details see Rousseau and Blais 1985), GIST (see Cunha and Sousa 2002), TRACS II (see Fores et al. 2001), TURNI (see Kroon and Fischetti 2001) and DOPT (Duty Optimization for Public Transport, see Huisman 2004).

Further solution approaches can be found in Wren (2004), Daduna and Wren (1988), Fores (1996) and Shen (2001); for a description of the terminology used in Bus and Driver Scheduling, the reader is referred to the glossary of Hartley (1981).

The remaining of the paper is organized as follows. In Sect. 1 a new mathematical model is provided for a BDSP under special constraints imposed by Italian transportation rules. Section 2 describes the general GRASP framework for solving combinatorial optimization problems and Sect. 3 contains a brief literature review on GRASP for transportation problems and states how it is applied to the special variant of the Bus Driver Scheduling Problem we studied here. In Sect. 4, we report computational results; in particular, in Sect. 4.2 we compare the GRASP procedure and the approach of Huisman et al. (2005), using the same random instances. Finally, in Sect. 5 we close the article with suggestions for future work.

1 Problem description and mathematical formulation

In this section, a new mathematical model is provided for a BDSP under special constraints originated by our collaboration with PluService Srl, a leading Italian group in software for transportation companies. The goal has been to model specific collective agreements and labour rules stated by the Italian government but also to generalize the state-of-the-art problem including constraints for a variety of trade-union rules and regulations for transportation companies.

Obviously, given the large number of different types of constraints, we have modelled only the most important of them such as working time, spreadover, idle time and constraints on depots (i.e., drivers must terminate the shifts at the same depot of departure). To the best of our knowledge, this is the first attempt of this kind to model such trade-union rules and regulations. The approaches utilized in literature are based on knapsack or multi-knapsack problems with additional constraints or on Set Partitioning formulations.

As mentioned before, Crew Scheduling Problems are among the most important problems that public transport companies must solve. It consists of assigning the Pieces-Of-Work (i.e., a sequence of trips, deadheads, and breaks between two relief points on a running board) to shifts in such a way that:

1. each Piece-Of-Work is performed by only one driver;
2. the shifts are feasible (that depends on a given set of working rules);
3. total operational costs of the shifts or the total number of shifts are minimized or both.

To determine the Pieces-Of-Work, it is necessary to solve a Vehicle Scheduling Problem. Let J be a set of Pieces-Of-Work, L be a set of positions (i.e., the sequence position of Pieces-Of-Work covered within a shift), and K be a set of shifts. The following binary variables are defined:

for each $j \in J$, $k \in K$, and $l \in L$,

$$x_{jkl} = \begin{cases} 1 & \text{if the Piece-Of-Work } j \text{ is in shift } k \text{ at position } l, \\ 0 & \text{otherwise;} \end{cases}$$

for each $k \in K$ and $l = 1, \dots, L - 1$,

$$y_{kl} = \begin{cases} 1 & \text{if there is some idle time in shift } k \\ & \text{immediately after position } l, \\ 0 & \text{otherwise;} \end{cases}$$

for each $i, j \in J$,

$$v_{ij} = \begin{cases} 1 & \text{if the Piece-Of-Work } j \text{ follows the Piece-Of-Work } i \text{ in a shift,} \\ 0 & \text{otherwise;} \end{cases}$$

for each i and $j \in J$,

$$u_{ij} = \begin{cases} 1 & \text{if the Pieces-Of-Work } i \text{ and } j \text{ are respectively the first} \\ & \text{and the last Piece-Of-Work in the same shift,} \\ 0 & \text{otherwise.} \end{cases}$$

Moreover, we define the following nonnegative variables:

for each $k \in K$,

TS_k = starting time of shift k ,

TE_k = ending time of shift k ,

for each $k \in K$ and $l = 1, \dots, L - 1$,

r_{kl} = the difference between the starting time of the next Piece-Of-Work
and the ending time of the Piece-Of-Work at position l in shift k .

Define now the following constant parameters:

- C_1 : maximum spreadover allowed (duration between the start and the end of a shift);
- C_2 : if between two consecutive Pieces-of-Work in a shift there is a delay larger than C_2 , then an idle time occurs;
- C_3 : maximum number of occurrences of idle time in a shift;
- C_4 : maximum total working time allowed;

and for each $j \in J$,

- ts_j and te_j denote, respectively, the start and end time of Piece-Of-Work j ;
- R_j is equal to 1 if there is idle time in Piece-Of-Work j , 0 otherwise;
- d_j is the working time of Piece-Of-Work j ;
- dep_j is equal to 1 if Piece-Of-Work j starts from the depot, 0 otherwise;

and for each $i \in J$ and $j \in J$,

- s_{ij} is equal to 2 if Piece-Of-Work j can follow Piece-Of-Work i , 1 otherwise;
- t_{ij} is equal to 2 if Piece-Of-Work i starts from the depot and Piece-Of-Work j ends in the same depot, 1 otherwise.

Furthermore, for each pair of Pieces-Of-Work (i, j) , let $w_{ij} \geq 0$ be the associated cost to carry out Piece-Of-Work j directly after Piece-Of-Work i , where $w_{ij} = +\infty$ if the pair (i, j) is not compatible or if $i = j$. Similarly, for each pair of Pieces-Of-Work (i, j) that starts and ends in the same depot, let W_{ij} be the nonnegative cost incurred when a driver starts his shift with Piece-of-Work i and terminates the shift with Piece-of-Work j . Moreover, let $c_j \geq 0$ be the associated costs to carry out a shift.

Depending on the choice of the cost coefficient, different objectives can be modelled. The number of shifts is minimized if $c_j = 1$ for each Piece-Of-Work j that starts from a depot, and $w_{ij} = 0$ and $W_{ij} = 0$ for each compatible pair (i, j) . Instead, if $c_j = 0$ for each Piece-Of-Work j and quantities W_{ij} and w_{ij} are proportional to the operational costs, including penalties for idle times, then the objective function minimizes overall operational costs. Moreover, any combination of the previous two objective functions can be modelled using specific choices of the values c_j , W_{ij} and w_{ij} .

The Bus Driver Scheduling Problem can be formulated as follows:

$$\min \sum_{k=1}^K \sum_{j=1}^J c_j x_{jk1} + \sum_{i=1}^J \sum_{j=1}^J w_{ij} v_{ij} + \sum_{i=1}^J \sum_{j=1}^J W_{ij} u_{ij}$$

subject to

$$\sum_{j=1}^J x_{jkl} \leq 1, \quad \forall k = 1, \dots, K, \quad \forall l = 1, \dots, L, \quad (1)$$

$$\sum_{j=1}^J x_{jkl+1} \leq \sum_{j=1}^J x_{jkl}, \quad \forall k = 1, \dots, K, \quad \forall l = 1, \dots, L-1, \quad (2)$$

$$\sum_{j=1}^J te_j x_{jkl} \leq \sum_{j=1}^J ts_j x_{jkl+1} + MQ \left(1 - \sum_{j=1}^J x_{jkl+1} \right), \quad \forall k = 1, \dots, K, \quad \forall l = 1, \dots, L-1, \quad (3)$$

$$TE_k \geq \sum_{j=1}^J te_j x_{jkl}, \quad \forall k = 1, \dots, K, \quad \forall l = 1, \dots, L, \quad (4)$$

$$TS_k = \sum_{j=1}^J ts_j x_{jk1}, \quad \forall k = 1, \dots, K, \quad (5)$$

$$TE_k - TS_k \leq C_1, \quad \forall k = 1, \dots, K, \quad (6)$$

$$\sum_{j=1}^J ts_j x_{jkl+1} - \sum_{j=1}^J te_j x_{jkl} \leq C_2 + My_{kl},$$

$$\forall k = 1, \dots, K, \forall l = 1, \dots, L-1, \quad (7)$$

$$\sum_{j=1}^J \sum_{l=1}^L R_j x_{jkl} + \sum_{l=1}^{L-1} y_{kl} \leq C_3, \quad \forall k = 1, \dots, K, \quad (8)$$

$$\sum_{l=1}^L \sum_{j=1}^J d_j x_{jkl} + \sum_{l=1}^{L-1} r_{kl} \leq C_4, \quad \forall k = 1, \dots, K, \quad (9)$$

$$r_{kl} \leq MQ \left(\sum_{j=1}^J x_{jkl+1} \right), \quad \forall k = 1, \dots, K, \forall l = 1, \dots, L-1, \quad (10)$$

$$r_{kl} \leq \sum_{j=1}^J ts_j x_{jkl+1} - \sum_{j=1}^J te_j x_{jkl} + My_{kl} + MQ \left(1 - \sum_{j=1}^J x_{jkl+1} \right),$$

$$\forall k = 1, \dots, K, \forall l = 1, \dots, L-1, \quad (11)$$

$$r_{kl} \geq \sum_{j=1}^J ts_j x_{jkl+1} - \sum_{j=1}^J te_j x_{jkl} - My_{kl} - MQ \left(1 - \sum_{j=1}^J x_{jkl+1} \right),$$

$$\forall k = 1, \dots, K, \forall l = 1, \dots, L-1, \quad (12)$$

$$\sum_{k=1}^K \sum_{l=1}^L x_{jkl} = 1, \quad \forall j = 1, \dots, J, \quad (13)$$

$$x_{ikl} + x_{jkl+1} \leq 1 + v_{ij}, \quad k = 1, \dots, K, \forall l = 1, \dots, L-1,$$

$$\forall j = 1, \dots, J, \forall i = 1, \dots, J, \quad (14)$$

$$1 + v_{ij} \leq s_{ij}, \quad \forall i = 1, \dots, J, \forall j = 1, \dots, J, \quad (15)$$

$$x_{ik1} + x_{jkl} \leq t_{ij} + \left(\sum_{\tilde{j}=1}^J x_{\tilde{j}kl+1} \right),$$

$$\forall k = 1, \dots, K, \forall l = 1, \dots, L-1,$$

$$\forall i = 1, \dots, J, \forall j = 1, \dots, J, \quad (16)$$

$$x_{ik1} + x_{jkl} \leq 1 + u_{ij} + \left(\sum_{\tilde{j}=1}^J x_{\tilde{j}kl+1} \right),$$

$$\forall k = 1, \dots, K, \forall j = 1, \dots, J,$$

$$\forall l = 1, \dots, L-1, \forall i = 1, \dots, J, \quad (17)$$

$$\begin{aligned}
 x_{ik1} + x_{jkl} &\leq t_{ij}, \quad \forall i = 1, \dots, J, \forall j = 1, \dots, J, \\
 \forall k &= 1, \dots, K,
 \end{aligned}
 \tag{18}$$

$$\begin{aligned}
 x_{jk1} &\leq dep_j, \quad k = 1, \forall k = 1, \dots, K, \forall j = 1, \dots, J, \\
 x_{jkl}, y_{kl} &\in \{0, 1\}, \quad \forall j = 1, \dots, J, \forall k = 1, \dots, K, \forall l = 1, \dots, L, \\
 v_{ij}, u_{ij} &\in \{0, 1\}, \quad \forall i = 1, \dots, J, \forall j = 1, \dots, J, \\
 TE_k &\geq 0, \quad TS_k \geq 0, \quad r_{kl} \geq 0, \quad \forall k = 1, \dots, K, \forall l = 1, \dots, L,
 \end{aligned}
 \tag{19}$$

where M and MQ are given constant parameters. The objective function minimizes the total number of shifts and/or the total operational costs. Constraints (1) impose that at most a single Piece-Of-Work j is in position l in shift k . Constraints (2) impose that, if there is a Piece-Of-Work at position $l + 1$, then there is a Piece-Of-Work at position l ; constraints (3) assert that the ending time of Piece-Of-Work j in shift k at position l must not exceed the starting time of Piece-Of-Work j in shift k at position $l + 1$.

Constraints (4), along with (5) and (6), impose limits on the total time between the starting and the ending time of the shift (i.e. the spreadover). Constraints (7) limit the number of breaks between Pieces-Of-Work and constraints (8) impose an upper bound on the number of occurrences of idle times. Constraints (9)–(12) impose a limit on the total duration of the journey in any shift (i.e. working time). In particular, constraints (10)–(12) impose that r_{kl} is exactly the rest time between positions l and $l + 1$ in shift k unless there is idle time. Constraints (13) ensure that each Piece-Of-Work is covered exactly once.

Constraints (14) and (15) impose compatibility between Pieces-Of-Work (two Pieces-Of-Work i and j are compatible if and only if they can be executed consecutively by the same driver, i.e. j can immediately follow i : $te_i + \tau_{ij} \leq ts_j$). Constraints (16)–(19) assert that the starting depot of any shift k must be the same as the ending one.

1.1 Computational results

The exact solutions for the mathematical model introduced in this subsection have been obtained by using (GAMS 2005) (General Algebraic Modelling System) and Cplex 9.1.2. Tables 1 and 2 show the computational results for our model. The first column contains the total number of Pieces-Of-Work; then, for each problem type we indicate three different objective functions (minimization of total number of shifts, of total operational costs, and finally a combination of both). In Table 1, for 16, 19, and 25 Pieces-Of-Work we report the optimal solution cost, the first integer feasible solution cost, and the running times needed to find them. In Table 2 we report the first integer feasible solution cost for 30, 38, and 50 Pieces-Of-Work. Note that, for 70 Pieces-Of-Work, we report the first integer feasible solution only for the minimization of the total number of shifts, since in the other cases the model is not able to find a solution.

The results obtained show that, unfortunately, the exact model can only be used to solve small or medium size instances of the problem. Given the high computational

Table 1 Experimental results for our mathematical model on 16, 19, and 25 Pieces-Of-Work

Problem	Objective Function	Optimal Cost	Time (in sec.)	First Feasible Solution Cost	Time (in sec.)
16 POWs	min. shift	5.00	1.40	5.00	0.86
	min. costs	2680.00	443.32	2710.00	0.95
	min. comb	2685.00	575.33	2715.00	0.87
19 POWs	min. shift	6.00	2.07	6.00	1.17
	min. costs	3150.00	630.32	3150.00	1.44
	min. comb	3156.00	999.06	3186.00	1.11
25 POWs	min. shift	8.00	117.97	8.00	1.90
	min. costs	4060.00	6239.26	4090.00	1.67
	min. comb	4068.00	65598.96	4098.00	1.78

Table 2 Experimental results for our mathematical model on 30, 38, 50, and 70 Pieces-Of-Work

Problem	Objective Function	First Feasible Solution Cost	Time (in sec.)
30 POWs	min. shift	10.00	2.51
	min. costs	4475.00	2.48
	min. comb	4485.00	2.52
38 POWs	min. shift	12.00	4.90
	min. costs	5615.00	5.54
	min. comb	5627.00	4.94
50 POWs	min. shift	17.00	6.41
	min. costs	8420.00	6.50
	min. comb	8437.00	6.49
70 POWs	min. shift	23.00	344242.92
	min. costs	—	—
	min. comb	—	—

intractability of the problem, feasible solutions of good quality for large instances can be obtained in a reasonable computational time only by applying a heuristic technique.

2 Greedy randomized adaptive search procedure

Greedy Randomized Adaptive Search Procedure (GRASP) is a constructive multi-start metaheuristic which has been applied to a wide range of well known combinatorial optimization problems, including academic and industrial problems in scheduling, routing, logic, partitioning, location and layout, graph theory, assignment, manufacturing, transportation, telecommunications, electrical power systems, and VLSI

```

algorithm grasp(MaxIter,  $\alpha$ ,  $c$ )
1   $x_b := \emptyset$ ;
2  for  $k = 1$  to MaxIter do
3     $x := \text{ConstructGreedyRandomizedSolution}(\alpha)$ ;
4     $\bar{x} := \text{LocalSearch}(x)$ ;
5    if ( $c'\bar{x} < c'x_b$ ) then  $x_b := \bar{x}$ ;
6  endfor
7  return( $x_b$ );
end grasp

```

Fig. 3 General GRASP procedure for a minimization problem

design (see, for example, Feo and Resende 1989, 1995; Festa and Resende 2002, 2009a, 2009b).

At each iteration, GRASP produces a new feasible solution x and applies a local search procedure to identify in $\mathcal{N}(x)$ (the neighborhood of the current point x) a better feasible solution. Each GRASP iteration consists of two phases (see Fig. 3):

1. a construction phase, where a feasible solution is produced;
2. a local search phase, where a local optimum in the neighborhood of the current solution is sought.

The basic GRASP construction phase is similar to the semi-greedy heuristic proposed by Hart and Shogan (1987).

Since experience shows that the better the feasible solution x is the better the neighbor solution $\bar{x}$ and the output solution x_b results, the construction phase of a GRASP procedure (line 3) aims at finding a good starting solution x (for the local search). This is achieved by adopting a randomized and adaptive greedy strategy. Starting from an empty solution, a new candidate element to the current partial solution is added until a complete feasible solution is obtained. The choice of which new element to add is determined by the order of all candidate elements in a candidate list J . This order reflects a prefixed *greedy criterion* based on the myopic “benefit” connected to the selection of that particular element. However, this benefit is not constant, but *adaptive*, and it is updated at each iteration of the construction phase to take into account the previous selected elements. The probabilistic component of the construction phase is in the random choice of one of the best candidates in the list; therefore, not necessarily the best candidate will be added. The set of these good candidates is called *Restricted Candidate List (RCL)*; clearly $RCL \subseteq J$ and within the *RCL* an element is randomly selected. In Fig. 3, as stopping criterion, the reaching of a predefined maximum number of iterations MaxIter is used, similarly to other meta-heuristics.

3 GRASP for the bus driver scheduling problem

The use of GRASP for the BDSP has been already proposed in literature. Souza et al. (2003), for instance, have suggested a hybrid approach to solve School Timetabling

Problems, that uses a greedy randomized construction phase to obtain a feasible solution, which is then followed by a Tabù Search procedure.

Sosnewska (2000), instead, describes two heuristic procedures based on Simulated Annealing and GRASP for a simplified Fleet Assignment Problem. Both heuristics are based on swapping parts of sequence of flight legs assigned to an aircraft (rotation cycle) between two randomly chosen aircrafts.

Argüello et al. (1997) have proposed a neighborhood search technique that uses a different approach for constructing the initial feasible solution. Moreover, two types of partial route exchange operations are proposed. The first operation exchanges flight sequences with identical endpoints; in the second operation, the exchanged sequence of flights must have the same origination airport, but the termination airports are swapped.

The first attempt to apply GRASP to the BDSP, that is the core of our investigation, is done by Lourenço et al. (2001). They have used GRASP not only as a solution method to solve the BDSP, but also as a procedure inside a multi-objective Tabù Search and Genetic Algorithms. In the construction phase, they use a greedy criterion based on the costs associated with the shifts. The local search procedure utilizes a 1-exchange neighborhood. The computational results are analyzed according to the different meta-heuristics applied to real instances. These methods have been successfully incorporated in the GIST Planning Transportation Systems. In the next subsection, we describe GRASP for BDSP applied to our model.

3.1 GRASP construction phase

The GRASP construction phase builds a feasible schedule T , i.e., a set of feasible shifts T_k , by selecting n_k compatible Pieces-Of-Work, one at a time, until all Pieces-Of-Work have been assigned.

The restricted candidate list depends on a parameter α that is randomly selected in the interval $[0, 1]$ and this value is not changed during the construction phase.

The value of α reflects the ratio of randomness versus greediness in the construction process. A value $\alpha = 1$ corresponds to a pure random selection, i.e., between all compatible Pieces-Of-Work we select in random way a Piece-Of-Work to be added to the partial solution, whereas $\alpha = 0$ leads to a pure greedy selection, i.e., between all compatible Pieces-Of-Work we select a candidate for which the difference between its starting time and the ending time of previous Piece-Of-Work is minimum.

The solution T is initially empty and the set J of candidate Pieces-Of-Work is the set of all Pieces-Of-Work. A candidate list is constructed for each Piece-Of-Work according to pairwise compatibility.

Initially, we sort the Pieces-Of-Work by the time of departure, and then we select the j -th ($1 \leq j \leq |J| - 1$) Piece-Of-Work to be added to the solution. The restricted candidate list RCL includes all Pieces-Of-Work in the candidate set J with time $t_j \leq \underline{t} + \alpha(\bar{t} - \underline{t})$, where

$$\underline{t} = \min_j \{ts_{j+1} - te_j\} \quad \text{and} \quad \bar{t} = \max_j \{ts_{j+1} - te_j\}.$$

Then, a Piece-Of-Work $j \in RCL$ is randomly selected and added to the partial solution. Once the Piece-Of-Work j is selected, the set of Pieces-Of-Work must be adjusted to take into account that j is now part of the current solution and must be removed from the current set of candidates.

The construction procedure is *adaptive* because, starting from the partial solution, each time a new Piece-Of-Work is added, we rebuild the RCL, taking into account the characteristics of the specific element added to the current partial solution. The set of remaining candidates may change, because some Pieces-Of-Work may be not anymore compatible with the new choice.

Once the current shift is completed, a new shift is constructed using the same steps. The updating procedure for the candidate list is the computational bottleneck of the construction phase. The procedure ends when all Pieces-Of-Work have been assigned.

3.2 GRASP local search phase

Starting from the feasible solution obtained in the construction phase, the local search procedure is applied. In this phase a neighborhood of the current solution is defined and a better trial solution is searched.

The neighborhood structures depend on the problem to be solved and fitting efficient neighborhood functions that with high probability lead to high quality local optima can be viewed as one of the major challenges of local search procedure design. Often, for the same problem several different neighborhood functions have been proposed and tested to experimentally validate their efficiency.

For the BDSP, we have defined and used three different neighborhoods: a 1-swap neighborhood, a variable neighborhood swap, and an extension of the latter. In the first case, we only interchange compatible Pieces-Of-Work, while the variable neighborhood swap is based on the interchange of compatible partial shifts (a partial shift is a sequence of compatible Pieces-Of-Work).

3.2.1 1-swap neighborhood

A feasible solution $T = \{T_1, T_2, \dots, T_N\}$ corresponds to a set of N shifts with $N = |K|$, where the shift T_k is a set of r_k Pieces-Of-Work (POWs):

$$T = \bigcup_{k=1}^N T_k, \quad T_k = \bigcup_{j=1}^{r_k} POW_j.$$

Given $T = \{T_1, T_2, \dots, T_N\}$ we define a neighborhood $\mathcal{N}(T)$ as the set of feasible neighbor solutions $\bar{T}$ that can be obtained selecting two shifts $T_k, T_i \in T$, two Pieces-Of-Work $POW_h \in T_k$ and $POW_l \in T_i$, and interchanging the assignment of the two Pieces-Of-Work, i.e. $\bar{T}_k = T_k \setminus POW_h \cup POW_l$, while $\bar{T}_i = T_i \setminus POW_l \cup POW_h$. In order to preserve feasibility of the neighbor solution generated so far, POW_l and POW_h have to be compatible.

Formally, $\mathcal{N}(T)$ is defined as follows:

$$\begin{aligned}\mathcal{N}(T) = \{ \bar{T} = \{ \bar{T}_1, \bar{T}_2, \dots, \bar{T}_N \} \mid & \forall i = 1, 2, \dots, N-1, \\ & \forall k = 1, 2, \dots, N, \quad k > i, \\ & \forall s = 1, 2, \dots, N, \\ & \forall l = 1, 2, \dots, r_i, \\ & \forall h = 1, 2, \dots, r_k, \\ & \bar{T}_s = T_s \quad \forall s \neq i, k \\ & \bar{T}_k = T_k \setminus POW_h \cup POW_l, \\ & \bar{T}_i = T_i \setminus POW_l \cup POW_h \}.\end{aligned}$$

Note that, the above defined neighborhood $\mathcal{N}(T)$ is a variant of the classical swap neighborhood where we exchange compatible Pieces-Of-Work, and shifts with better costs can thus be found. However, this technique does not reduce the number of shifts, since each element in $\mathcal{N}(T)$ is still made of N shifts.

3.2.2 Variable neighborhood swap

An alternative neighborhood structure is the Variable Neighborhood Swap. Given a feasible solution $T = \{T_1, T_2, \dots, T_N\}$, we define a neighborhood $\mathcal{N}_1(T)$ as the set of feasible neighbor solutions $\bar{T}$ that can be obtained selecting two shifts $T_k, T_i \in T$, $k, i \in \{1, 2, \dots, N = |K|\}$, $k \neq i$, and interchanging two sequences of Pieces-Of-Work not necessarily of the same length but preserving compatibility. In particular, we decompose T_k in partial shifts, each having h_k and r_k Pieces-Of-Work, respectively. In more detail, assuming that $h_k \leq r_k$, T_k is decomposed as follows:

$$T_{k1} = \bigcup_{j=1}^{h_k} POW_j, \quad T_{k2} = \bigcup_{j=h_k+1}^{r_k} POW_j.$$

A similar decomposition can be applied to $T_i \in T$, $i \neq k$ to obtain T_{i1} and T_{i2} and then the interchange is performed as shown in Fig. 4.

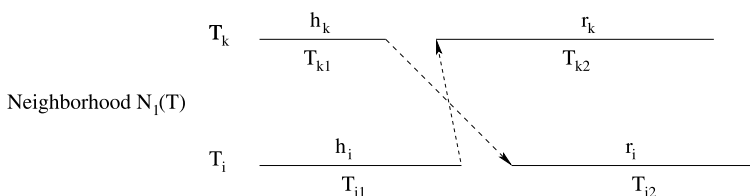


Fig. 4 Interchange operation for the variable neighborhood swap

Formally, the neighborhood structure is defined as follows:

$$\begin{aligned}\mathcal{N}_1(T) = \{ \overline{T} = \{ \overline{T}_1, \overline{T}_2, \dots, \overline{T}_N \} \mid & \forall i = 1, 2, \dots, N-1, \\ & \forall i < k \leq N, \\ & \forall s = 1, 2, \dots, N, \\ & \forall i1 = 1, 2, \dots, h_i, \\ & \forall i2 = h_i + 1, \dots, r_i, \\ & \forall k1 = 1, 2, \dots, h_k, \\ & \forall k2 = h_k + 1, \dots, r_k, \\ & \overline{T}_s = T_s, \quad \forall s \neq i, k, \\ & \overline{T}_k = T_{k1} \cup T_{i2}, \\ & \overline{T}_i = T_{i1} \cup T_{k2} \}.\end{aligned}$$

Unlike the 1-swap neighborhood, this technique not only guarantees the possibility of constructing shifts of lower cost, but can also reduce the number of shifts.

In exploring this neighborhood, the following two cases are of particular interest:

$$\begin{aligned}h_i = 0, \quad h_k = r_k & \implies T_{i1} = \emptyset, \quad T_{k2} = \emptyset, \\ \overline{T}_k = T_{k1} \cup T_{i2} &= T_k \cup T_i, \\ \overline{T}_i &= \emptyset; \\ h_i = r_i, \quad h_k = 0 & \implies T_{i2} = \emptyset, \quad T_{k1} = \emptyset, \\ \overline{T}_i = T_{i1} \cup T_{k2} &= T_i \cup T_k, \\ \overline{T}_k &= \emptyset.\end{aligned}$$

In both cases, shift i and shift k are merged in a single shift (while feasibility is still satisfied).

Moreover, the following cases correspond to the construction of feasible shifts by combining original shifts in different ways.

$$\begin{aligned}0 < h_i < r_i, \quad 0 < h_k < r_k & \implies \overline{T}_i = T_{i1} \cup T_{k2}, \quad \overline{T}_k = T_{k1} \cup T_{i2}, \\ & T_{k1} = \emptyset, \\ 0 < h_i < r_i, \quad h_k = 0 & \implies \overline{T}_i = T_{i1} \cup T_{k2} = T_{i1} \cup T_k, \\ & \overline{T}_k = T_{i2}, \\ & T_{k2} = \emptyset, \\ 0 < h_i < r_i, \quad h_k = r_k & \implies \overline{T}_i = T_{i1} \cup T_{k2}, \\ & \overline{T}_k = T_{k1} \cup T_{i2} = T_k \cup T_{i2},\end{aligned}$$

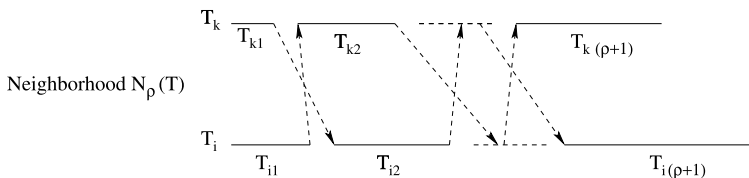


Fig. 5 Interchange operations for the variable neighborhood swap with $\rho > 1$

$$\begin{aligned}
 & h_i = 0, \quad 0 < h_k < r_k \implies \begin{aligned} & T_{i1} = \emptyset, \\ & \overline{T}_i = T_{k2}, \\ & \overline{T}_k = T_{k1} \cup T_{i2} = T_{k1} \cup T_i, \end{aligned} \\
 & h_i = r_i, \quad 0 < h_k < r_k \implies \begin{aligned} & T_{i2} = \emptyset, \\ & \overline{T}_i = T_{i1} \cup T_{k2} = T_i \cup T_{k2}, \\ & \overline{T}_k = T_{k1}. \end{aligned}
 \end{aligned}$$

In general, if we decompose T_k in $\rho + 1$ partial shifts as follows:

$$T_{k1} = \bigcup_{j=1}^{h1_k} POW_j, \quad T_{k2} = \bigcup_{j=h1_k+1}^{h2_k} POW_j, \dots, T_{k\rho+1} = \bigcup_{j=h\rho_k+1}^{n_k} POW_j,$$

with $h1_k \leq h2_k \leq \dots \leq h(\rho - 1)_k \leq h\rho_k \leq n_k$ and apply the same decomposition also to T_i , then a set of interchanges can be performed as shown in Fig. 5.

Formally, the neighborhood structure is defined as follows:

$$\begin{aligned}
 \mathcal{N}_\rho(T) &= \{\overline{T} = \{\overline{T}_1, \overline{T}_2, \dots, \overline{T}_N\} \mid \forall i = 1, 2, \dots, N - 1, \\
 & \quad \forall i < k \leq N, \\
 & \quad \forall i1 = 1, 2, \dots, h1_i, \\
 & \quad \forall i2 = h1_i + 1, \dots, h2_i, \\
 & \quad \vdots \\
 & \quad \forall i\rho = h(\rho - 1)_i + 1, \dots, h\rho_i, \\
 & \quad \forall k1 = 1, 2, \dots, h1_k, \\
 & \quad \forall k2 = h1_k + 1, \dots, h2_k, \\
 & \quad \vdots \\
 & \quad \forall k\rho = h(\rho - 1)_k + 1, \dots, h\rho_k, \\
 & \quad \overline{T}_s = T_s \quad \forall s = 1, 2, \dots, N, \quad s \neq i, k \quad (20) \\
 & \quad \overline{T}_k = T_{k1} \cup T_{i2} \cup T_{k3} \cup \dots \cup T_{i\rho} \cup T_{k\rho+1}, \\
 & \quad \forall \rho = 2l, \quad \forall l \in \mathbb{N} \setminus \{0\}, \quad (21)
 \end{aligned}$$

$$\begin{aligned}\bar{T}_i &= T_{i1} \cup T_{k2} \cup T_{i3} \cup \dots \cup T_{k\rho} \cup T_{i\rho+1}, \\ \forall \rho &= 2l, \forall l \in \mathbb{N} \setminus \{0\},\end{aligned}\quad (22)$$

$$\begin{aligned}\bar{T}_k &= T_{k1} \cup T_{i2} \cup T_{k3} \cup \dots \cup T_{k\rho} \cup T_{i\rho+1}, \\ \forall \rho &= 2l + 1, \forall l \in \mathbb{N},\end{aligned}\quad (23)$$

$$\begin{aligned}\bar{T}_i &= T_{i1} \cup T_{k2} \cup T_{i3} \cup \dots \cup T_{i\rho} \cup T_{k\rho+1}, \\ \forall \rho &= 2l + 1, \forall l \in \mathbb{N}\}.\end{aligned}\quad (24)$$

Note that, for $\rho = 1$, the neighborhood $\mathcal{N}_\rho(T)$ is exactly the neighborhood $\mathcal{N}_1(T)$ previously defined (see p. 455). For $\rho > 1$, $\mathcal{N}_\rho(T)$ can be easily obtained from $\mathcal{N}_{\rho-1}(T)$. In fact,

- when ρ is even, $T_{k\rho+1}$ and $T_{i\rho+1}$ are the last partial shifts of $\bar{T}_k$ (21) and $\bar{T}_i$ (22), respectively;
- when ρ is odd, $T_{i\rho+1}$ and $T_{k\rho+1}$ are the last partial shifts of $\bar{T}_k$ (23) and $\bar{T}_i$ (24), respectively.

4 Computational results

In this section, we report computational results for real-world problems and comparisons between the solution produced by an exact method based on Set Covering and by the GRASP procedure. The exact method we used solves the problem in two steps: at first it solves the Vehicle Scheduling Problem using a formulation similar to the one proposed by Löbel (1998), generating a set of running boards (Vehicle Schedule). Then, the running boards are divided, generating the Pieces-of-Work. The Pieces-Of-Work are combined to obtain feasible shifts, using a k -decision tree: as k increases, the number of generated shifts increases, too. The upper bound for the number of generated shifts is fixed to 2 millions. Once the feasible shifts have been generated, we solve a classical Set-Covering problem using Cplex 9.1. The parameter k controls the total number of columns (feasible shifts generated). Starting from a partial shift we construct k new partial shifts by adding to the current shift a new Piece-of-Work. This procedure is iterated until a feasible shift is obtained and the corresponding column is added. In this k -branches tree, trimming occurs (and therefore the corresponding column is not added) if the cost for the shift is above a pre-specified limit. In this way, only a small subset of “good” shifts are included in the optimization problem.

With respect to the model provided in Sect. 1, for this Set Covering formulation and the GRASP method a more complex objective function is used to take more precisely into account some quality requirements of the solution. For example, a specific cost can be assigned to each generated feasible shift, we consider, in addition to the quantities already included in the model in Sect. 1, costs related to the effective length of the idle times and breaks and costs related to extra working time and overspread.

All numerical tests were carried out on a Pentium 4 with CPU 3.20 GHz and with 1.00 GB RAM. Both exact method and GRASP procedure were coded in C and compiled with DEV-C++ (2005) version 5.0 beta 9.2.

The real data sets have been provided by the PluService Srl. The results are summarized in Table 3. For each problem instance, we report for both algorithms the total number of trips, the value of the objective function, the total number of shifts, the total number of vehicles (running boards), the total number of the Pieces-Of-Work, and the time in seconds for obtaining the solution. We set the GRASP MaxIter parameter equal to 1000. For each algorithm, the last column reports the average efficiency of the schedule, i.e., a measure of the relative efficiency calculated as the mean value of working times per spreadover unit. Note that some problem instances could not be solved exactly while for some others the special character (*) indicates that the exact method was not able to find the optimal solution in a reasonable computational time. For such instances, Table 3 shows the first feasible integer solution found by the exact method using a smaller value of k .

4.1 Probability distribution of solution time in GRASP for BDSP

In this subsection, we describe a procedure recently proposed by Aiex et al. (2007) to create time-to-target solution value plots for measured CPU times that are assumed to fit a shifted exponential distribution. This is often the case in local search based heuristics for combinatorial optimization, such as Simulated Annealing, Genetic Algorithms, Iterated Local Search, Tabù Search, WalkSAT, and GRASP. Such plots are used to compare algorithms for combinatorial optimization problems; in particular, this technique has been used for comparing different heuristics, the same algorithm on different instances, parallel implementation using several number of processors or parallelization strategies, algorithms using different strategies and an algorithm using different parameter settings (for more details the reader can refer to Aiex et al. 2007).

The hypothesis is that the run times fit a two parameter exponential distribution. We consider two test problems out of the 16 that we have solved and we measure the time needed to find a value of the objective function which is equal or better than a given target value. The analysis has been performed using a target value (the best value produced by GRASP performing 1000 iterations). The first set is made of 76 trips and the target value is 15438086, while the second of 104 trips and the target value is 13917766. We run the GRASP procedure for $n = 100$ independent times for each instance/target pair with different seed for the number generator.

Following Aiex et al. (2000), we compare the empirical and theoretical distributions using a standard graphical methodology for data analysis based on Chambers et al. (1983).

For each instance, we sort in increasing order the running times t_i to which a probability $p_i = (i - \frac{1}{2})/n$ is associated and we draw the points $z_i = (t_i, p_i)$, $i = 1, \dots, n$.

A theoretical Q-Q plot is obtained by drawing the quantiles of an empirical distribution against the quantiles of a theoretical distribution of the data. In other words, we first sort the data in increasing order and then we calculate the quantiles of the theoretical exponential distribution and finally we draw the data against the theoretical quantiles.

To test if the Q-Q plots follow the progress of the line in Fig. 6, we apply a variability information to each point. More precisely, we calculate an interval for the

Table 3 Computational results obtained by an exact method and by GRASP on real-world BDSP problems provided by PluService Srl

Problem	Method	Cost	n^o of shifts	n^o of vehicles	n^o of POWs	time in sec.	eff. %
24 trips	exact	5228983	4	3	27	0.77	74.43
	grasp	5370930	4	3	27	0.00	81.50
60 trips(*) ($k = 10$)	exact	13384164	9	3	65	124.80	68.83
	grasp	8541870	7	5	65	0.00	45.43
62 trips	exact	14662692	14	12	74	2.98	56.25
	grasp	15412000	14	12	74	1.00	56.25
63 trips	exact	14712334	13	13	76	2.55	54.74
	grasp	14846189	13	13	76	1.00	72.92
69 trips	exact	24372774	15	12	81	2.17	61.80
	grasp	27799160	16	12	81	0.00	51.88
72 trips	exact	12859590	12	12	84	0.22	40.11
	grasp	17002600	12	12	84	0.00	48.25
104 trips	exact	15811450	15	33	114	3090.72	73.77
	grasp	18303800	17	33	114	5.00	69.00
110 trips(*) ($k = 5$)	exact	17902000	17	9	119	1655.19	65.62
	grasp	16189400	14	9	119	4.00	73.21
136 trips	exact	–	–	5	141	–	–
	grasp	15038200	14	5	141	4.00	100.00
142 trips(*) ($k = 3$)	exact	43012665	26	23	165	949.47	66.29
	grasp	30108700	27	23	165	13.00	61.37
150 trips(*) ($k = 5$)	exact	43097989	28	24	173	214.31	69.04
	grasp	30915000	29	24	173	14.00	60.76
153 trips(*)	exact	–	–	22	175	–	–
	grasp	30582000	28	22	175	22.00	66.64
153 trips(*) ($k = 5$)	exact	26162329	25	20	173	740.97	57.73
	grasp	28294400	26	20	173	19.00	59.20

Table 3 (Continued)

Problem	Method	Cost	n^o of shifts	n^o of vehicles	n^o of POWs	time in sec.	eff. %
158 trips(*)	exact	39467120	25	21	179	403.36	68.18
	($k = 3$)						
	grasp	27187800	25	21	179	3.00	60.72
161 trips(*)	exact	65751000	44	34	195	19738.39	50.79
	($k = 10$)						
	grasp	45454000	43	34	195	59.00	64.11

estimate z_i , $i = 1, \dots, n$ of the standard deviation in the vertical direction from the line fitted to the plot. In Fig. 6 we draw the Q-Q plot with this additional variability information.

Finally, in Fig. 6 we show a superimposed plot of the empirical and theoretical distributions. All analysis can be executed by using the Perl language program (see for more detail Aiex et al. 2007) to create time-to-target solution plots for measured run times.¹ With regards the set of data made of 76 trips and target value equal to 15438086, we conclude that the probability of the heuristic finding a solution at least as good as the target value in at most 100 seconds is about 70%, in at most 150 seconds is about 80%, and in at most 200 seconds is about 90%. Moreover, for the second set of data made of 104 trips and target value is 13917766, the probability of finding a solution at least as good as the target value in at most 1000 seconds is about 60%, in at most 1500 seconds is about 75%, and in at most 2000 seconds is about 90%.

4.2 A numerical comparison for random data instances

Huisman et al. (2005)² provided two different models and algorithms (based on a combination of column generation and Lagrangian relaxation) for the integration of the Vehicle and Crew Scheduling in the Multiple-Depot case, and compare those integrated approaches with the traditional sequential approach.

They consider two types of instances (1 and 2), which differ in the travel speed. As the travel speed increases, trips are shorter and vehicles as well as drivers will cover more trips.

In their article, they considered six cases: 3 instances, where there are four lines (from A to B, A to C, A to D and B to C) and 3 instances with five lines (adding to the four lines previously seen a line from C to E). All these points are relief locations.

Huisman et al. (2005) have tested the algorithms on 10 instances randomly generated (10, 20 and 40 trips per line/direction with four and five lines). Therefore, the total number of trips ranges from 80 to 400. The 2 depots case has been executed for

¹The Perl program can be downloaded from <http://www.research.att.com/~mgcr/ttplots>.

²We use the random data instances available at <http://www.few.eur.nl/few/people/huisman/instances.htm>.

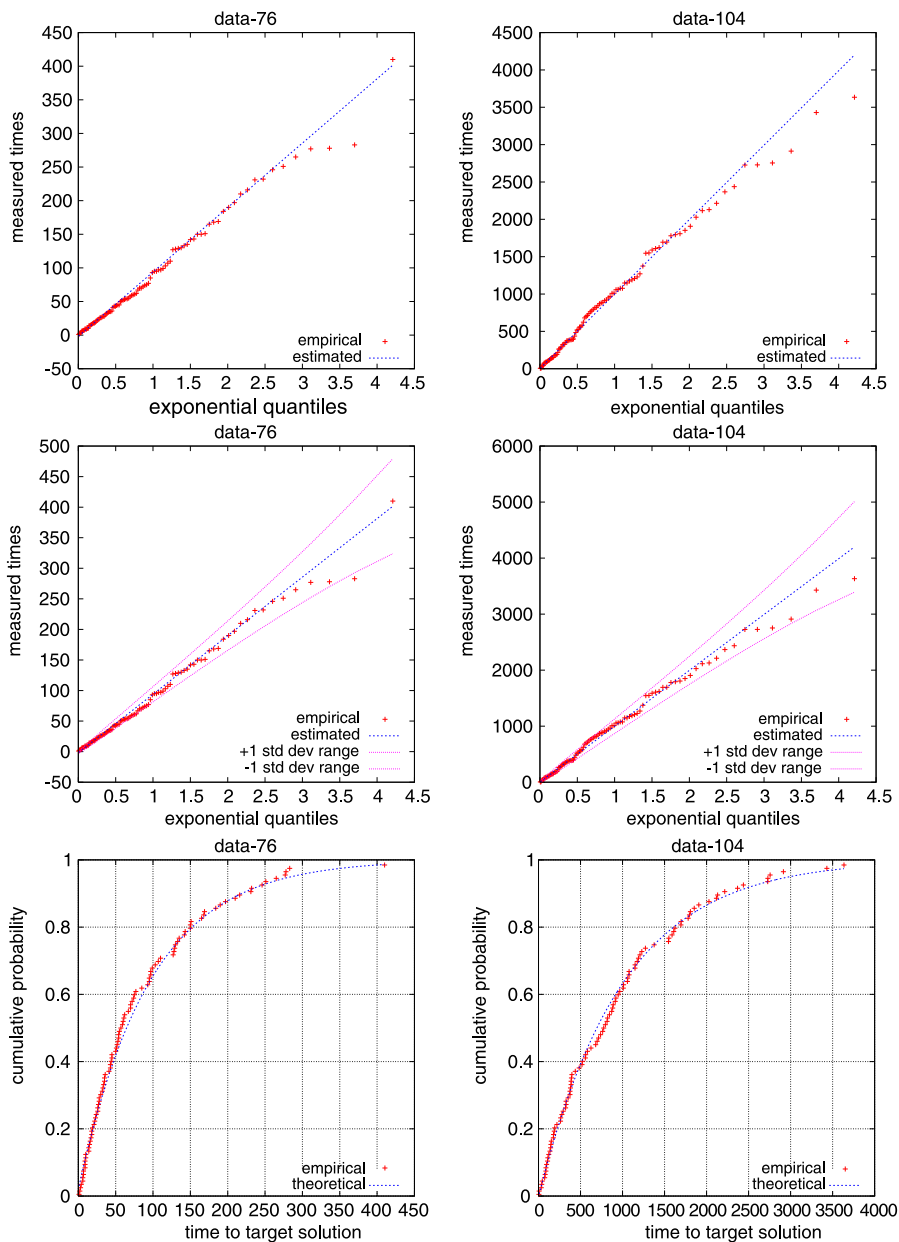


Fig. 6 Probability distribution of solution time in GRASP for BDSP

all the instances of both 1 and 2, while the 4 depots case has not been executed for the 320 and 400 trips instances.

In Tables 4–5 and 8–9, we report the results obtained in Huisman et al. (2005) for instances of both type with 2 and 4 depots, respectively, for the traditional sequential

Table 4 Average results random data instances—2 depots—type 1

	trips	80	100	160	200	320	400
seq.	vehicles	9.2	11.0	14.8	18.4	26.7	32.9
	drivers	23.8	29.0	35.9	44.5	60.8	74.9
	total	33.0	40.0	50.7	62.9	87.5	107.8
int. 1	vehicles	9.2	11.0	14.8	18.4	26.7	32.9
	drivers	20.6	24.8	33.5	40.7	60.1	73.2
	total	29.8	35.8	48.3	59.1	86.8	106.1
int. 2	vehicles	9.2	11.0	14.8	18.4	26.7	32.9
	drivers	20.6	24.6	33.5	41.0	60.0	74.2
	total	29.8	35.6	48.3	59.4	86.7	107.1

Table 5 Average results random data instances—2 depots—type 2

	trips	80	100	160	200	320	400
seq.	vehicles	11.3	13.8	19.3	24.4	35.8	44.2
	drivers	26.9	32.9	44.4	54.7	79.0	96.8
	total	38.2	46.7	63.7	79.1	114.8	141.0
int. 1	vehicles	11.3	13.8	19.3	24.4	35.8	44.2
	drivers	24.9	29.1	42.3	51.4	77.8	95.0
	total	36.2	42.9	61.6	75.8	113.6	139.2
int. 2	vehicles	11.3	13.8	19.3	24.4	35.8	44.2
	drivers	24.7	29.1	42.6	52.2	78.0	95.6
	total	36.0	42.9	61.9	76.6	113.8	139.8

approach and the two integrated approaches. We report the number of trips, the average number of vehicles, the average number of drivers and the sum of both vehicles and drivers.

We also have used these same random instances in our sequential approach. First, we have solved the Vehicle Scheduling Problem producing a set of feasible vehicle schedules and then, we have solved the Crew Scheduling Problem with pure GRASP, producing the feasible shift schedules. While Huisman et al. (2005) have analyzed five different shift types, the feasible shift schedules we have generated are sets of Pieces-Of-Work with no particular properties; our working time corresponds to 6 hours and a half, while the spreadover is 12 hours.

Tables 6–7 and 10–11 show the results we have obtained when applying pure GRASP to the same random instances used by Huisman et al. (2005) with 2 and 4 depots, respectively. Here, we see that the average number of vehicles is in essence the same as in Huisman et al. (2005) for both types 1 and 2, and both in case of 2 and 4 depots. When analyzing the first case we see an increase of 0.1 percent in two instances out of six, while in the second case we obtain a decrease of 0.2 percent in one instance out of six and this result holds both in case of 2 and 4 depots.

Table 6 Average results
random data instances—2
depots—type 1

	trips	80	100	160	200	320	400
seq.	vehicles	9.3	11.0	14.8	18.5	26.7	32.9
	drivers	20.3	23.7	30.7	37.6	53.6	65.7
	total	29.6	34.7	45.5	56.1	80.3	98.6
	time	17.0	15.0	53.6	71.1	157.2	204.0
	total time	33.7	43.8	117.7	143.2	366.9	458.9

Table 7 Average results
random data instances—2
depots—type 2

	trips	80	100	160	200	320	400
seq.	vehicles	11.3	13.6	19.3	24.4	35.8	44.2
	drivers	22.3	26.7	37.1	45.8	67.1	83.4
	total	33.6	40.3	56.4	70.2	102.9	127.6
	time	14.9	18.2	45.1	63.0	127.9	267.6
	total time	29.8	41.9	104.0	141.1	325.6	455.9

Table 8 Average results
random data instances—4
depots—type 1

	trips	80	100	160	200
seq.	vehicles	9.2	11.0	14.8	18.4
	drivers	25.8	29.9	38.8	47.1
	total	35.0	40.9	53.6	65.5
int. 1	vehicles	9.2	11.0	14.8	18.4
	drivers	20.5	25.3	34.1	41.6
	total	29.7	36.3	48.9	60.0
int. 2	vehicles	9.2	11.0	14.8	18.4
	drivers	20.4	25.2	34.7	42.0
	total	29.6	36.2	49.5	60.4

As far as the average number of drivers is concerned, we obtain different results due to the fact that, both in case 1 and 2, with 2 or 4 depot, respectively, we generate shifts of a different type. Looking at Tables 6–7 and 10–11, in case of 4 depots and for instances with less than 160 trips, the average number of drivers is less than or equal to the average number of drivers in case of 2 depots.

Tables 6–7 and 10–11 contain also information about running times in seconds. All tables report for all input instances the mean time (time) needed to find the better solution and the mean time (total time) needed to run a total of 1000 iterations. Looking at the results, our GRASP finds the output solution in at most 50% out of the total running time.

Table 9 Average results random data instances—4 depots—type 2

	trips	80	100	160	200
seq.	vehicles	11.3	13.8	19.3	24.4
	drivers	28.3	34.1	45.9	56.8
	total	39.6	47.9	65.2	81.2
int. 1	vehicles	11.3	13.8	19.3	24.4
	drivers	25.1	30.3	42.9	52.1
	total	36.4	44.1	62.2	76.5
int. 2	vehicles	11.3	13.8	19.3	24.4
	drivers	24.8	30.0	44.0	53.6
	total	36.1	43.8	63.3	78.0

Table 10 Average results random data instances—4 depots—type 1

	trips	80	100	160	200
seq.	vehicles	9.3	11.0	14.8	18.5
	drivers	19.0	23.7	30.4	38.9
	total	28.3	34.7	45.2	57.4
	time	25.4	22.1	68.6	55.1
	total time	55.3	52.4	129.4	163.7

Table 11 Average results random data instances—4 depots—type 2

	trips	80	100	160	200
seq.	vehicles	11.3	13.6	19.3	24.4
	drivers	22.2	26.3	36.5	46.6
	total	33.5	39.9	55.8	71.0
	time	25.4	23.0	82.5	78.6
	total time	48.4	48.4	121.5	149.2

5 Conclusions and future works

This paper deals with the problem of determining the best scheduling for Bus Drivers, i.e., with a $\mathcal{NP}$ -hard problem consisting of finding the minimum number of drivers to cover a set of Pieces-Of-Work subject to a variety of rules and regulations that must be enforced such as the spreadover and the working time. A new mathematical model has been formulated for a special variant of the problem faced by Italian transportation companies that have to satisfy particular collective agreements and labour rules. Since this model can only be usefully applied to small or medium size problem instances, to solve large problem instances, a Greedy Randomized Adaptive Search Procedure (GRASP) has been proposed and numerical results carried out on real-world BDSP instances provided by PluService Srl show the effectiveness of the designed metaheuristic approach.

Since recent literature has shown that GRASP benefits from the combination with other meta-heuristics (e.g. Festa et al. 2002), as future work we are planning without adding much additional computational burden to design new hybrid heuristics that combine GRASP with other modern and efficient meta-heuristics, such as VNS (Variable Neighborhood Search) and PR (Path-Relinking).

Acknowledgement The authors would like to thank the anonymous reviewers for their comments and suggestions which have been revealed useful to improve both quality and readability of the manuscript.

References

- Aiex, R.M., Resende, M.G.C., Ribeiro, C.C.: Probability distribution of solution time in GRASP: An experimental investigation. *J. Heuristics* **8**, 200–212 (2000)
- Aiex, R.M., Resende, M.G.C., Ribeiro, C.C.: TTTplots: a Perl program to create time-to-target plots. *Optim. Lett.* **1**, 355–366 (2007)
- Argüello, M., Bard, J., Yu, G.: A GRASP for aircraft routing in response to groundings and delays. *J. Combin. Optim.* **1**, 211–228 (1997)
- Carraresi, P., Nonato, M., Girardi, L.: Network models, Lagrangian relaxation and subgradients bundle approach in crew scheduling problems. In: Daduna, J.R., Branco, I., Paixão, J.R. (eds.) *Computer-Aided Transit Scheduling*, Lisbon, pp. 187–212. Springer, Berlin (1993)
- Chambers, J.M., Cleveland, W.S., Kleiner, B., Tukey, P.A.: *Graphical Methods for Data Analysis*. Chapman & Hall, London (1983)
- Cunha, J.F., Sousa, J.P.: The bus stops here—GIST: A decision support system problem. *OR/MS Today* **27**(2), 324–335 (2002)
- Curtis, S.D., Smith, B.M., Wren, A.: Forming bus driver schedules using constraint programming. Technical Report, University of Leeds, School of Computer Studies, Report 99.05 (1999)
- Daduna, J.R., Wren, A.: *Computer-Aided Transit Scheduling*. Lecture Notes in Economics and Mathematical Systems, vol. 308. Springer, Heidelberg (1988)
- Daduna, J.R., Branco, I., Paixão, J.M.P.: *Computer-Aided Transit Scheduling*. Lecture Notes in Economics and Mathematical Systems, vol. 430. Springer, Heidelberg (1995)
- Desaulniers, G., Hickman, M.D.: Public transit. Technical Report, Les Cahiers du GERAD, G-2003-77 (2003)
- Desrochers, M., Rousseau, J.M.: *Computer-Aided Transit Scheduling*. Lecture Notes in Economics and Mathematical Systems, vol. 386. Springer, Heidelberg (1992)
- Desrochers, M., Soumis, F.: A column generation approach to the urban transit crew scheduling problem. *Transp. Sci.* **23**(1), 1–13 (1989)
- DEV-C++: <http://www.bloodshed.net/dev/index.html> (2005)
- Dias, T.G., Sousa, J.P., Cunha, J.F.: A genetic algorithm for the bus driver scheduling problem. In: 4th Metaheuristics International Conference (2001)
- Dias, T.G., Sousa, J.P., Cunha, J.F.: A multiobjective genetic algorithm for the bus driver scheduling problem. In: The 6th Metaheuristics International Conference (2005)
- Feo, T.A., Resende, M.G.C.: A probabilistic heuristic for a computationally difficult set covering problem. *Oper. Res. Lett.* **8**, 67–71 (1989)
- Feo, T.A., Resende, M.G.C.: Greedy randomized adaptive search procedures. *J. Glob. Optim.* **6**, 109–133 (1995)
- Festa, P., Resende, M.G.C.: GRASP: An annotated bibliography. In: Ribeiro, C.C., Hansen, P. (eds.) *Essays and Surveys in Metaheuristics*, pp. 325–367. Kluwer Academic, Dordrecht (2002)
- Festa, P., Resende, M.G.C.: An annotated bibliography of GRASP—Part I: algorithms. *Int. Trans. Oper. Res.* **16**(1), 1–24 (2009a)
- Festa, P., Resende, M.G.C.: An annotated bibliography of GRASP—Part II: applications. *Int. Trans. Oper. Res.* **16**(2), 131–172 (2009b)
- Festa, P., Pardalos, P., Resende, M., Ribeiro, C.: Randomized heuristics for the max-cut problem. *Optim. Methods Softw.* **7**, 1033–1058 (2002)
- Fischetti, M., Martello, S., Toth, P.: The fixed job schedule problem with spread-time constraints. *Oper. Res.* **35**, 849–858 (1987)

- Fischetti, M., Martello, S., Toth, P.: The fixed job schedule problem with working-time constraints. *Oper. Res.* **37**, 395–403 (1989)
- Fores, S.: Column generation approaches to bus driver scheduling. Ph.D. Thesis, The University of Leeds, School of Computer Studies (1996)
- Fores, S., Proll, L., Wren, A.: Experiences with a flexible driver scheduler. In: Voß, S., Daduna, J.R. (eds.) *Computer-Aided Scheduling of Public Transport*, pp. 137–152. Springer, Berlin (2001)
- Fores, S., Proll, L., Wren, A.: TRACS II: a hybrid IP/heuristic driver scheduling system for public transport. *J. Oper. Res. Soc.* **53**, 1093–1100 (2002)
- GAMS: General Algebraic Modelling System. <http://www.gams.com/docs/gams/document.html> (2005)
- Hart, J., Shogan, A.: Semi-greedy heuristics: an empirical study. *Oper. Res. Lett.* **6**, 107–114 (1987)
- Hartley, T.: A glossary of terms in bus and crew scheduling. In: Wren, A. (ed.) *Computer Scheduling of Public Transport*, pp. 353–359. North-Holland, Amsterdam (1981)
- Huisman, D.: Integrated and dynamic vehicle and crew scheduling. Ph.D. Thesis, Erasmus Universiteit Rotterdam (2004)
- Huisman, D., Freling, R., Wagelmans, A.P.M.: Multiple-depot integrated vehicle and crew scheduling. *Transp. Sci.* **39**(4), 491–502 (2005)
- Kroon, L., Fischetti, M.: Crew scheduling for Netherlands railways destination: customer. In: Voß, S., Daduna, J.R. (eds.) *Computer-Aided Scheduling of Public Transport*, pp. 181–201. Springer, Berlin (2001)
- Löbel, A.: Vehicle scheduling in public transit and Lagrangian pricing. *Oper. Res.* **44**(12), 1637–1649 (1998)
- Lourenço, H., Paixão, J., Portugal, P.: Multiobjective metaheuristics for the bus-driver scheduling problem. *Transp. Sci.* **35**, 331–343 (2001)
- Mesquita, M., Paías, A., Respício, A.: Branching approaches for integrated vehicle and crew scheduling. *Public Transp.* **1**, 21–37 (2009)
- Moz, M., Respício, A., Pato, M.V.: Bi-objective evolutionary heuristics for bus driver rostering. *Public Transp.* **1**, 189–210 (2009)
- Portugal, R., Lourenço, H., Paixão, J.: Driver scheduling problem modelling. *Public Transp.* **1**, 103–120 (2009)
- Rousseau, J.M.: *Computer Scheduling of Public Transport 2*. North-Holland, Amsterdam (1985)
- Rousseau, J.M., Blais, J.Y.: Hastus: An interactive system for buses and crew scheduling. In: Rousseau, J.M. (ed.) *Computer Scheduling of Public Transport 2*, pp. 45–60. North-Holland, Amsterdam (1985)
- Rousseau, J.M., Desrosiers, J.: Results obtained with crew-opt: a column generation method for transit crew scheduling. In: Daduna, J.R., Branco, I., Paixão, J.R. (eds.) *Computer-Aided Transit Scheduling*, Lisbon, pp. 349–358. Springer, Berlin (1993)
- Shen, Y.: Tabù search for bus and train driver scheduling with time windows. Ph.D. Thesis, The University of Leeds, School of Computing (2001)
- Sosnewska, D.: Optimization of a simplified fleet assignment problem with metaheuristics: simulated annealing and GRASP. In: Pardalos, P.M. (ed.) *Approximation and Complexity in Numerical Optimization: Continuous and Discrete Problems*, pp. 477–488. Kluwer Academic, Dordrecht (2000)
- Souza, M., Maculan, N., Ochi, L.: A GRASP-tabù search algorithm for school timetabling problems. In: Resende, M., Sousa, J. (eds.) *Metaheuristics: Computer Decision-Making*, pp. 659–672. Kluwer Academic, Dordrecht (2003)
- Voß, S., Daduna, J.: *Computer-Aided Transit Scheduling*. Lecture Notes in Economics and Mathematical Systems, vol. 505. Springer, Heidelberg (2001)
- Wilson, N.H.M.: *Computer-Aided Transit Scheduling*. Lecture Notes in Economics and Mathematical Systems, vol. 471. Springer, Heidelberg (1999)
- Wren, A.: *Computer Scheduling of Public Transport*. North-Holland, Amsterdam (1981)
- Wren, A.: Scheduling vehicles and their drivers—forty years' experience. Technical Report, University of Leeds, School of Computing Research Report Series, Report 2004.03 (2004)
- Wren, A., Rousseau, J.M.: Bus driver scheduling problem—an overview. In: Daduna, J., Branco, I., Paixão, J. (eds.) *Computer-Aided Transit Scheduling*. Lecture Notes in Economics and Mathematical Systems, vol. 430, pp. 173–187. Springer, Heidelberg (1995)